

Analyze Our Findings

1. Capture ICMP Protocol

Field	Analysis
Protocol	ICMP (Internet Control Message Protocol)
Purpose	To verify network connectivity between the Kali Linux VM and the Windows 10 VM using Echo Request and Echo Reply messages (ping).
Source IP	192.168.32.130 (Kali Linux VM)
Destination IP	192.168.32.136 (Windows 10 VM)
Packet Count	The status bar shows 324 ICMP packets displayed (filtered from 566 total packets). The capture contains multiple Echo Requests and Echo Replies.
Key Observations	• Successful communication between the Kali and Windows VMs. • Echo Request packets were sent from 192.168.32.130 to 192.168.32.136. • Echo Reply packets were received from 192.168.32.136 back to 192.168.32.130. • The ping output shows a TTL value of 128, which is typical for Windows operating systems. • The round-trip time ranged from approximately 0.5 ms to 2.4 ms, indicating low network latency within the virtual lab. • No packet loss is visible in the terminal output, indicating reliable connectivity. • Wireshark correctly identified the traffic as ICMP and displayed both request and reply packets.

Report:

ICMP Traffic Analysis

The ICMP protocol was analyzed by generating ping traffic from the Kali Linux virtual machine (192.168.32.130) to the Windows 10 virtual machine (192.168.32.136). Wireshark successfully captured the ICMP Echo Request and Echo Reply packets, confirming bidirectional communication between the two systems. The ping responses showed a TTL value of 128 and round-trip times between approximately 0.5 ms and 2.4 ms, indicating stable connectivity with low latency. A total of 324 ICMP packets were displayed after applying the ICMP filter, demonstrating successful packet capture and analysis within the virtual environment.

2. DNS Traffic Analysis

Field	Analysis
Protocol	DNS (Domain Name System)
Purpose	To resolve the domain name google.com into its corresponding IP address(es). DNS translates human-readable domain names into IP addresses that computers use for communication.
Source IP	192.168.32.136 (Kali Linux VM sending the DNS query)
Destination IP	192.168.32.2 (DNS server configured for the virtual machine)
Packet Count	The capture contains 22 total packets , including DNS queries, DNS responses, and a few ARP packets. The displayed DNS packets show multiple query/response exchanges.
Key Observations	• The nslookup google.com command generated DNS query packets. • The configured DNS server is 192.168.32.2 (shown in the terminal). • Wireshark captured both DNS Query and DNS Response packets. • The DNS server successfully returned multiple IPv4 addresses for google.com (e.g., 142.250.118.100, 142.250.118.102, 142.250.118.113, 142.250.118.139, 142.250.118.101, 142.250.118.138) and an IPv6 address (2404:6800:4013:813::71). • DNS communication uses UDP (source port is a temporary client port, destination port 53). • The successful responses confirm that domain name resolution is functioning correctly in the virtual lab.

Report:

DNS Analysis

DNS traffic was generated using the nslookup google.com command from the Kali Linux virtual machine. Wireshark captured the DNS query and corresponding response packets exchanged with the configured DNS server (192.168.32.2). The

server successfully resolved the domain name and returned multiple IPv4 and IPv6 addresses for google.com, confirming proper DNS functionality within the virtual environment. Packet analysis showed the use of UDP transport with destination port 53, which is the standard protocol and port used for DNS communication.

3. HTTP Traffic Analysis

Field	Analysis
Protocol	HTTP (Hypertext Transfer Protocol)
Purpose	HTTP is used to request and transfer web pages between a client (browser) and a web server. In this lab, it was used to retrieve the Example Domains webpage.
Source IP	192.168.32.130 (Kali Linux VM)
Destination IP	172.66.147.243 (Web server hosting the requested page)
Packet Count	After applying the http filter, 2 HTTP packets are displayed: one HTTP GET request and one HTTP 200 OK response .
Key Observations	• The browser sent an HTTP GET request to retrieve the webpage. • The destination server responded with HTTP/1.1 200 OK, indicating the request was successful. • The client IP is 192.168.32.130 and the server IP is 172.66.147.243. • The HTTP request packet size is 436 bytes, and the response packet size is 758 bytes. • HTTP data is transmitted in plain text, which means the contents can be inspected by packet analysis tools like Wireshark. • This demonstrates the request-response communication model used by the HTTP protocol.

Browser Activity

You opened the browser and visited the Example Domains website. The page loaded successfully, generating HTTP traffic that Wireshark captured.

Packet 147 – HTTP GET Request

- **Source:** 192.168.32.130
- **Destination:** 172.66.147.243
- **Method:** GET

This packet represents your browser asking the web server for the webpage.

Packet 159 – HTTP Response

- **Source:** 172.66.147.243
- **Destination:** 192.168.32.130
- **Status:** HTTP/1.1 200 OK

The server responded successfully and sent the webpage back to the browser

Report:

HTTP Analysis

HTTP traffic was generated by accessing a test webpage from the Kali Linux virtual machine. Wireshark captured an HTTP GET request from the client (192.168.32.130) to the web server (172.66.147.243), followed by an HTTP/1.1 200 OK response. The successful exchange demonstrates the HTTP request-response model. Because HTTP transmits data without encryption, packet contents can be inspected directly, highlighting the importance of HTTPS for secure web communication.

4. TLS (HTTPS) Traffic Analysis

Field	Analysis
Protocol	TLS (Transport Layer Security) – TLSv1.2 and TLSv1.3
Purpose	TLS encrypts communication between a client and a server, ensuring confidentiality, integrity, and authentication of transmitted data. HTTPS uses TLS to secure web traffic.
Source IP	192.168.32.130 (Kali Linux VM)
Destination IP	Multiple Google servers, including 34.160.144.191, 151.101.105.91, 104.18.24.232, and 104.20.23.154.
Packet Count	After applying the tls filter, 345 TLS packets are displayed from a total capture of 2,263 packets .
Key Observations	• The browser established encrypted HTTPS connections with Google servers. • Both TLS 1.2 and TLS 1.3 packets are visible in the capture. • Multiple application data packets indicate encrypted communication after the TLS session was established. • A TLSv1.3 Server Hello packet is visible, showing the server responding during the TLS handshake. • Packet details include the Transport Layer Security section, confirming successful TLS negotiation. • Unlike HTTP, the application data is encrypted and cannot be read directly in Wireshark without the appropriate session keys.

Browser Activity

You visited:

<https://www.google.com>

Since Google uses HTTPS, the browser automatically created encrypted TLS sessions.

Wireshark Filter

You correctly applied:

tls

This displays only TLS traffic, making it easier to analyze encrypted communications.

Source and Destination

Your Kali Linux VM (192.168.32.130) communicates with several Google servers.

This is normal because Google's infrastructure uses multiple servers for services such as search, content delivery, and security.

Report:

TLS/HTTPS Analysis

HTTPS traffic was generated by accessing <https://www.google.com> from the Kali Linux virtual machine. Wireshark captured the TLS handshake and encrypted application data exchanged between the client (192.168.32.130) and multiple Google servers. Both TLS 1.2 and TLS 1.3 were observed, demonstrating modern secure communication. Unlike HTTP, the application data remained encrypted, illustrating how TLS protects confidentiality and integrity during web browsing.

5. TCP Traffic Analysis

Field	Analysis
Protocol	TCP (Transmission Control Protocol)
Purpose	TCP provides reliable, connection-oriented communication between two devices. It ensures that data is delivered accurately, in order, and without loss before applications such as HTTP or HTTPS exchange data.
Source IP	192.168.32.135 (Kali Linux VM)
Destination IP	91.189.91.98 (Web server)
Packet Count	The capture contains 11,392 TCP packets displayed from a total of 11,716 captured packets .
Key Observations	• The capture shows the complete TCP three-way handshake (SYN → SYN, ACK → ACK). • The client initiates the connection using a SYN packet. • The server responds with a SYN, ACK packet. • The client completes the handshake with an ACK packet. • After the connection is established, an HTTP GET request is transmitted. • The server responds with HTTP/1.1 204 No Content, confirming successful communication. • The connection is terminated gracefully using FIN, ACK followed by ACK packets. • The TCP session uses destination port 80, which is the standard port for HTTP.

1. TCP Three-Way Handshake

The first three packets are:

Packet 6

192.168.32.135 → 91.189.91.98
[SYN]

Meaning

The Kali Linux VM requests to establish a TCP connection.

Packet 7

91.189.91.98 → 192.168.32.135
[SYN, ACK]

Meaning

The server acknowledges the client's request and agrees to establish the connection.

Packet 8

192.168.32.135 → 91.189.91.98
[ACK]

Meaning

The client acknowledges the server's response.

At this point, the TCP connection has been successfully established.

Report:

TCP Analysis

TCP traffic was analyzed to observe the complete lifecycle of a client-server connection. Wireshark captured the TCP three-way handshake consisting of **SYN**, **SYN-ACK**, and **ACK** packets, establishing a reliable connection between the Kali Linux virtual machine (192.168.32.135) and the remote web server (91.189.91.98). After the connection was established, an HTTP GET request was sent, followed by an HTTP response (204 No Content). The session concluded with a graceful connection termination using **FIN**, **ACK** packets, demonstrating the reliability and connection management features of TCP.

6. ARP Traffic Analysis

Field	Analysis
Protocol	ARP (Address Resolution Protocol)
Purpose	ARP is used to resolve an IPv4 address to its corresponding MAC (hardware) address within a local network before communication can begin.
Source	VMware_bf:5c:81 (MAC address of one VM) and VMware_e9:49:11 (MAC address of another device on the virtual network).
Destination	Broadcast (ff:ff:ff:ff:ff:ff) for ARP requests and the requesting host for ARP replies.
Packet Count	The capture contains 2,000 total packets , with 44 ARP packets displayed after applying the arp filter.
Key Observations	• ARP Requests such as "Who has 192.168.32.2? Tell 192.168.32.136" were observed. • ARP Replies such as "192.168.32.2 is at 00:50:56:e9:49:11" successfully resolved the MAC address. • Several ARP Probe packets were captured, indicating devices checking whether an IP address is already in use before assigning it. • An ARP Announcement was also observed, where a device advertised its own IP-to-MAC mapping. • Broadcast ARP requests are normal because the sender initially does not know the destination MAC address.

Report:

ARP Analysis

ARP traffic was analyzed to observe how devices resolve IPv4 addresses to MAC addresses on the local network. Wireshark captured ARP Requests, ARP Replies, ARP Probes, and ARP Announcements exchanged between virtual machines. The capture

demonstrated successful IP-to-MAC address resolution, with broadcast requests followed by direct replies from the target device. No suspicious ARP behavior or evidence of ARP spoofing was observed during the analysis.

7. UDP Traffic Analysis

Field	Analysis
Protocol	UDP (User Datagram Protocol)
Purpose	UDP provides fast, connectionless communication with low overhead. It is commonly used by DNS, DHCP, VoIP, video streaming, and online gaming where speed is more important than guaranteed delivery.
Source IP	192.168.32.136 (Client)
Destination IP	192.168.32.2 (DNS Server)
Packet Count	The capture contains 1,090 total packets , with 30 UDP packets displayed after applying the udp filter.
Key Observations	• DNS queries are transmitted using UDP destination port 53. • The selected packet shows a DNS query from 192.168.32.136 to the DNS server 192.168.32.2. • The UDP header shows Source Port: 58707 and Destination Port: 53. • Multiple DNS query and response packets are visible. • The capture also contains mDNS (Multicast DNS) packets used for local network service discovery. • No TCP handshake is present because UDP is a connectionless protocol.

Selected Packet

The highlighted packet is a DNS query.

Source IP: 192.168.32.136

Destination IP: 192.168.32.2

Protocol: DNS (over UDP)

UDP Header

In the packet details pane, Wireshark shows:

User Datagram Protocol

Source Port: 58707

Destination Port: 53

This means:

- **58707** → Temporary (ephemeral) client port selected by the operating system.
- **53** → Standard DNS server port.

UDP Analysis

UDP traffic was analyzed to observe connectionless communication in the virtual lab. Wireshark captured DNS queries sent from the client (192.168.32.136) to the DNS server (192.168.32.2) using destination port 53. The selected packet used an ephemeral source port (58707) and demonstrated the lightweight nature of UDP, where data is transmitted without establishing a prior connection. The capture also included Multicast DNS (mDNS) traffic, illustrating local network service discovery.